Iscte - Instituto Universitário de Lisboa, Portugal

Binary Sentiment Classification of Amazon Food Product Reviews: A Comparative Analysis

Afonso Abreu, Afonso Claudino, André Marques

Abstract. This work presents a comprehensive comparative analysis of several methodologies for binary sentiment classification (positive or negative), applied to reviews of Amazon food products. The experimental framework explores the evolution of Natural Language Processing (NLP), ranging from traditional lexicon-based approaches (NRC EmoLex) and statistical machine learning (Naive Bayes, SVM, Logistic Regression and Random Forest) to state-of-the-art architectures, including Transformers (DistilBERT) and LLMs (GPT-4o). Particular attention was paid to preprocessing strategies, such as negation handling, and to vectorization techniques such as TF-IDF and Word2Vec. The results indicate that the SVM model, when combined with TF-IDF and negation handling, achieves a highly competitive performance-efficiency ratio, proving to be a robust alternative for low-resource training scenarios. However, the GPT-4o model obtained the best Macro F1-Score (0.93), with a very limited test set, closely followed by fine-tuned DistilBERT (0.92), demonstrating the ability of context-sensitive models to capture the linguistic nuances of consumer feedback. The trade-offs between computational cost and predictive accuracy across different modeling paradigms are highlighted, and we conclude that fine-tuned DistilBERT appears to be a more recommended model for the task. The work was carried out in person by all group members, so it is not possible to quantify the contribution of each member.

1 Introduction

Sentiment analysis is one of the most prominent areas of text mining, using Natural Language Processing (NLP) techniques to automatically classify the affective polarity of a body of text, usually positive, negative or neutral.

In the current context, e-commerce platforms account for around 20% of global sales volume, with leading examples such as eBay, Wish and Alibaba. Among these, Amazon stands out as the dominant platform, holding approximately 17% of the worldwide e-commerce market share. Maintaining this dominance requires continuous monitoring of customer satisfaction, a task for which sentiment analysis of product reviews proves particularly well suited.

This work proposes a comparative evaluation of multiple sentiment analysis approaches applied to a dataset of food product reviews from the Amazon platform. The approaches considered fall into five distinct categories: lexicon- and rule-based models, namely TextBlob, VADER and Stanza; transformer-based models, using the distilbert-base-uncased-finetuned-sst-2-english model; application of the EmoLex/NRC sentiment lexicon; training of classical machine learning models (SVM, Logistic Regression, Naive Bayes, Decision Trees and Random Forest) with text representations obtained via TF-IDF, Bag of Words and Word2Vec, complemented by the fine-tuned DistilBERT model; and instruction-based models, represented by OpenAI's GPT-4o.

2 Related Work

Gope et al. [1] presented a sentiment analysis study on Amazon product reviews, comparing five machine learning algorithms with a deep learning approach. In terms

of classical models, SVM, Random Forest, Multinomial Naïve Bayes, Bernoulli Naïve Bayes and Logistic Regression were evaluated, with Random Forest obtaining the best accuracy on the test set (0.92), with an F1-score of 0.95. The authors applied TF-IDF as the feature representation method and oversampled the minority classes to mitigate the imbalance of the dataset. For the deep learning approach, they implemented an RNN, achieving an accuracy of 0.98 and concluding that this model is the most suitable approach for the analysis. The work shows that deep learning-based methods clearly outperform classical classifiers when applied to sufficiently large datasets, but it does not consider unsupervised lexical approaches or pre-trained Transformer-based models.

Hawlader et al. [2] compared three feature extraction techniques (Bag of Words, TF-IDF and Word2Vec) crossed with six supervised models: SVM, Naïve Bayes, Decision Tree, Random Forest, Logistic Regression and Multilayer Perceptron (MLP). The study used a dataset of Amazon electronics product reviews, balanced with 24,000 examples per class. The results showed that MLP with BOW obtained the best overall accuracy (0.92), followed by SVM with TF-IDF (0.90). The Word2Vec representation was consistently inferior to the others, with accuracy below 0.71, which the authors attribute to the corpus being too small to adequately capture the semantic relationships between words. The work concludes that the choice of vectorization method has as significant an impact on final performance as the choice of classifier, and that dense approaches such as Word2Vec require larger volumes of data to be competitive.

Lu [3] presented a comparative study focused on the effectiveness of two text vectorization techniques, TF-IDF and Word2Vec, in the context of sentiment analysis, using the Amazon Fine Food Reviews dataset. After preprocessing the data, both representations were applied to classification models to evaluate their performance. The results showed that TF-IDF has a better ability to discriminate between sentiment classes, especially on larger datasets, while Word2Vec stands out in capturing semantic relationships between words. The author concludes that the two approaches have complementary characteristics, suggesting that they be combined to improve the overall performance of sentiment analysis systems.

Tabany et al. [4] presented a study of sentiment analysis applied to the detection of fake reviews on the Amazon platform. They also performed sentiment classification of the reviews, which is the part most related to our work. Several classifiers were evaluated, including SVM, Naïve Bayes, Logistic Regression and Random Forest, with the SVM model showing the best performance in the polarity classification task, reaching an accuracy of around 0.93. As regards fake review detection, they showed that linguistic and emotional patterns can also be useful in identifying them.

Ali et al. [5] carried out a comprehensive sentiment analysis study on Amazon reviews, comparing several models. For text preprocessing, they used techniques such as normalization and tokenization, followed by feature extraction using BOW and TF-IDF. The authors evaluated several traditional classifiers, including Naïve Bayes, Random Forest, Decision Tree and Logistic Regression. In addition, they explored deep learning models, such as CNN and BiLSTM, and Transformer models, namely BERT and XLNet.

The authors concluded that Transformer-based models achieve the best overall performance, with BERT reaching approximately 0.89 accuracy. Even so, the authors found that traditional approaches with TF-IDF continue to deliver competitive results at a lower computational cost.

Maurya and Pratap [6] developed a sentiment analysis system applied to Amazon product reviews, using classical supervised machine learning techniques. For sentiment classification, the authors used supervised algorithms such as Naïve Bayes and SVM, evaluating the ability of these models to distinguish between positive and negative reviews. The SVM model performed better, benefiting from its ability to find effective separating boundaries in high-dimensional spaces, a common characteristic of text classification tasks. The results showed that, even with traditional representations of the reviews, it is possible to obtain effective polarity classifications, demonstrating the robustness of classical supervised methods for sentiment analysis on e-commerce data.

3 Data Used

This study used the amazon_reviews corpus, split into training and test sets. The dataset comprises a total of 51,319 instances, the vast majority of which (48,902) belong to the training set. A marked class imbalance was observed, with 77% of the observations labeled as positive (77% in the training set and 69% in the test set).

Manual exploratory analysis, complemented by the word cloud visualization (Fig. 1), confirmed that the data domain centers on food products, with a high frequency of terms such as “flavor”, “taste”, “coffee” and “tea”. In addition, the presence of terms such as “dog” and “cat” shows that the corpus also covers the pet food segment.

We also found that many reviews contain HTML residue such as “
” or even URLs, which should not be considered in this work.

Fig. 1. Word Cloud of the dataset (stopwords and non-relevant words removed)



4 Work Carried Out

4.1 Dataset Cleaning

Regarding general preprocessing, we focused mainly on removing noise from the dataset, and for this purpose we used Regular Expressions to remove URLs, HTML, punctuation and numbers. This was the general cleaning used for most models, with some exceptions, which will be explicitly justified in the following section.

4.2 Application of Existing Models

Lexicon- and Rule-Based Models.

For the Baseline, we tested three models:

TextBlob. This model assigns a polarity to each review, from -1 to 1. [7] Since this work uses binary classification, we decided that reviews with polarity 0 should be classified as negative. Since we know that the dataset is heavily imbalanced towards positives, and since we have this prior knowledge about the composition of the test set, we considered that it would be bad practice to treat polarity 0 as positive, as we would be artificially improving the models' statistics, also considering that if a product review shows such an unclear signal of positive sentiment, it is closer to a negative review.

VADER. Since VADER benefits from the presence of punctuation [8], we did not remove it during preprocessing, and we kept preserving the structure of the text. As in the case of TextBlob, reviews with polarity 0 were classified as negative.

Stanza. The preprocessing was the same as for VADER. Stanza assigns the number 1 to reviews it predicts to be neutral.[9] Again, we assigned these reviews to the negative class.

Transformer-Based Models.

We used only one transformer-based model (without fine-tuning):

DistilBERT (SST2-english). No additional work was done to test this model other than keeping punctuation in the reviews, since it is a pre-trained model [10] and fine-tuning transformer-based models will be a different experiment. This specific model was chosen on the Professors' recommendation.

4.3 Sentiment Lexicon

Several experiments were carried out using only the NCR EmoLex lexicon [11], with the ultimate goal of summing the total polarity of all the words present both in a review and in the lexicon in order to classify it.

First, Case Folding was applied to the reviews, since the terms in the lexicon are also in lowercase. Even though we found that EmoLex is not lemmatized (or at least not fully, in several cases), which can be confirmed, for example, by the presence of “cheerful”, “cheerfulness”, “cheer” and “cheering”, we decided that we would have a better chance of creating “matches” between terms if they were lemmatized in the reviews, given that if the term “cheered” were present in a review, it would not be assigned any polarity value but, after being lemmatized, it would become “cheer”,

which is already in EmoLex, and it would be assigned positive polarity. For this, we used the WordNetLemmatizer from the NLTK library [12]. This function, however, requires a POS tag to lemmatize all terms that are not nouns (which is the default POS tag), so, for this purpose, we tokenized and performed POS tagging with the pos_tag function, also from NLTK. With this preprocessing, we carried out three experiments:

EmoLex without negation handling.

For each term in a review that was found in EmoLex with a positive tag, the value 1 would be added to the review's total polarity, while for terms with a negative tag, the value -1 was added. In this way, we indirectly also handle words that have both tags in the lexicon. Reviews with a total polarity value strictly greater than 0 are predicted as “positive”, the rest as “negative”.

EmoLex with negation handling (2-word scope).

In this case, the structure is identical to the previous case, with the addition of a system to handle negation. After lemmatization, each review becomes a list, which is ordered, made up of all the lemmatized terms. We create a set of negation terms, such as “no”, “barely”, “never”, “cannot”. Each review is a list (therefore ordered) containing all of its terms. Thus, the system goes through each term of a review, one by one. When a term belongs to the set of negation terms, we update the constant negation_active_until, which is equal to the index of the last negation term found plus 2, so that we have a way of keeping track, over the next two words in the list, of whether or not they are under the effect of negation. If the index of a term is equal to or less than the current value of negation_active_until, its polarity value is inverted, that is, if “good” is found up to two terms after a negation term, such as “not”, it takes on polarity -1, and if it is “bad” it takes on polarity 1. We decided to use this logic for negation, but we understand that “not good” carries a much more negative sentiment than “not bad” carries a positive one. We could, alternatively, have chosen to set cases like the latter to polarity 0.

EmoLex with negation handling (2-word scope) and valence shifting.

This case is the same as the previous one, but instead, in the case where a word with positive polarity is negated (“not good”), its new polarity value is now -1.27 times the old polarity value. We decided to carry out this experiment inspired by the results of Hogenboom et al. (2011) [13], who observed this to be the value that achieved the best results for negation on the movie review dataset on which they tested their models, even though these differ from ours.

4.4 Training of Machine Learning Models

Classical approaches.

After the lexicon-based approach, the use of supervised learning models was explored, in which training is performed on the training set, using the sckit-learn library [14].

In all the examples in this section, fit_transform was applied to the training set and only transform to the test set, for the BOW and TF-IDF vectorization.

With Multinomial Naïve Bayes, three main variations were tested:

Naïve Bayes with TF-IDF and Bag-of-Words (BoW).

To ensure a fair comparison between the absolute frequency of BOW and the relative frequency of TF-IDF, the vocabulary was limited to the 5000 most frequent words and standard English stopword removal was applied. The aim was to assess whether simple term counting would be sufficient to classify sentiment or whether weighting by term rarity would bring benefits in the food review domain.

Naïve Bayes with Bag-of-Words and Negation Handling,

To try to capture polarity inversion (one of the main limitations of BOW models), a third experiment was implemented using the mark_negation technique from the NLTK library. Unlike the previous experiments, the cleaning pipeline was restructured, with punctuation removal postponed until after the tokenization stage, since the negation algorithm depends on delimiters (such as periods and commas) to define the scope of the _NEG suffix.

In this configuration, stopwords were filtered in a customized way to ensure that negation particles (such as "not" and "never") were not removed before being processed. In addition, the feature limit was expanded to 10,000 features in order to accommodate the vocabulary growth generated by the new negated tokens (distinguishing good from good_NEG).

Continuing the exploration of supervised models, Logistic Regression was implemented, used here with the parameter class_weight='balanced' to mitigate the imbalance between the positive and negative classes in the dataset. Three different text representations were tested:

Logistic Regression with BOW and TF-IDF.

The first two experiments replicated the absolute-frequency BOW and weighted TF-IDF vectorizations with the 5,000-feature limit from Naive Bayes.

Logistic Regression with Word2Vec (Embeddings).

This experiment served to compare the performance of embeddings with the previous vectorizations. In this approach, the Word2Vec model [15] was trained specifically on the tokens of the training set reviews, using a dense vector architecture with 100 dimensions and a context window of 5 words.

Since Word2Vec generates vectors for individual words, the Average Pooling technique was used to represent each complete review: the mean vector of all terms present in the review that had a match in the trained model was computed.

Decision Tree with BOW and TF-IDF.

The Decision Tree was tested using the BOW and TF-IDF representations, mainly to test the performance of Random Forest. To mitigate the risk of overfitting, the maximum depth was fixed (max_depth=30). This restriction forces the model to select only the words with the highest information gain, ignoring overly specific details of the training set that might not generalize well to new data. class_weight='balanced' was also used, with the same justification as before.

Random Forest.

The Random Forest model was configured with 100 independent trees (n_estimators=100), operating through the Bagging technique. As in the previous experiment, these were applied to both BoW and TF-IDF vectors. class_weight='balanced' was used.

Finally, the use of Support Vector Machines was explored through the LinearSVC implementation. As in the previous models, the class_weight='balanced' weighting was applied to ensure that the decision boundary was not biased by the majority class. The SVM experiments were divided into four strands:

SVM with BoW and TF-IDF.

Using the 5,000-feature representations extracted by BOW and TF-IDF, the aim was to assess the ability of SVM to maximize the separation margin between positive and negative reviews. No measures different from the previous experiments were taken.

SVM with Word2Vec (Embeddings).

The SVM was trained on the same Word2Vec vectors (Average Pooling) as in the previous experiments.

SVM with TF-IDF and Negation Handling.

The most advanced experiment with this classifier involved integrating customized negation handling. The tokenization pipeline with mark_negation was used. For this task, the vocabulary was expanded to 15,000 features, allowing the SVM to treat negated terms (such as happy_NEG) as entirely distinct dimensions in the vector space.

Fine-tuning of a transformer-based model

DistilBERT (fine-tuned).

DistilBERT was then fine-tuned. DistilBERT was selected because it is a distilled, lighter and faster version of the original BERT, retaining approximately 97% of its performance. [16] Since DistilBERT is an encoder, it uses self-attention mechanisms. This allows the model to understand the meaning of a word based on the whole context of the sentence, both to the left and to the right. Consequently, negation handling becomes intrinsic to the model.

In terms of pipeline configuration, the Hugging Face transformers library was used to convert the dataset into a compatible format and the DistilBertTokenizer was applied. This process uses WordPiece tokenization, which breaks rare words down into subunits, mitigating the out-of-vocabulary word problem. The sequences were normalized through padding and truncation to ensure uniformity in the model inputs. In terms of fine-tuning, the distilbert-base-uncased model was trained for 3 epochs, using a learning rate = 2e-5 and a weight decay of 0.01 for regularization, values within the range recommended by its creators. We chose the not-yet-trained DistilBERT to compare with DistilBERT SST2, which had already been trained previously.

4.5 Use of Instruction-Based Models

Lastly, the LLM paradigm was explored, using the GPT-4o model through the iaedu.pt API. We aimed to assess how different levels of instruction and context influence the model's precision in Zero-shot and Few-shot Learning settings. To ensure statistical and computational feasibility, we used a balanced subset of 50 reviews (25 positive and 25 negative). This is a compromise with what it was possible to do with our resources, knowing that it will affect how comparable these experiments are with the others. Three prompting strategies were tested:

Generic Prompt (Zero-shot).

In this control approach, the model received a direct and minimalist instruction to classify the text only as “positive” or “negative”. This experiment served as a baseline to measure the model's intrinsic ability to understand sentiment without any additional context.

The prompt used was: *“You are a sentiment analyzer. Classify the review text as 'positive' or 'negative'.*
Respond ONLY with one of the words: positive or negative.”

Domain-Specialized Prompt (Zero-shot Domain).

To provide more context, a prompt was designed that assigns an expert persona to the model. The context was enriched with details specific to the Amazon food domain. This strategy aimed to test whether activating domain-specific knowledge in the model would result in greater sensitivity to critical nuances in the reviews.

The prompt used was: *“You are an expert in analyzing Amazon product reviews, especially food, beverages, supplements, and grocery products.*
Users write about taste, quality, price, delivery, texture, nutritional value, and overall satisfaction.
Classify the review below as 'positive' or 'negative' considering the context of purchasing food products on Amazon.
Respond ONLY with one of the words: positive or negative.”

Few-Examples Prompt (Few-shot Learning).

The most robust configuration involved including real examples drawn from the training set. Six balanced examples (3 positive and 3 negative) were selected, allowing the model to observe the expected classification pattern before processing the test review.

The prompt was: *“You are a sentiment analyzer specialized in Amazon reviews.*
Here are some classified examples:” (...) “Now classify the review below. Respond ONLY with 'positive' or 'negative'.”

5 Results Obtained

Table 1. Results of the Experiments

Model	Acc.	F1 Pos	F1 Neg	Macro F1
TextBlob	0.77	0.85	0.52	0.68
Vader	0.79	0.86	0.57	0.71
Stanza	0.63	0.67	0.57	0.62
DistilBERT SST2-english	0.82	0.86	0.76	0.81
EmoLex, tokenization, casefolding, POS, lemmatization	0.72	0.82	0.38	0.60
EmoLex, tokenization, casefolding, POS, lemmatization, negation (2-word scope)	0.72	0.81	0.49	0.65
EmoLex, tokenization, casefolding, POS, lemmatization, negation (2-word scope, valence shifting)	0.72	0.81	0.49	0.65
Naïve Bayes, cf, stopwords, TF-IDF, 5K features	0.78	0.86	0.50	0.68
Naïve Bayes, cf, stopwords, BOW, 5K features	0.85	0.89	0.75	0.82
Naïve Bayes, cf, stopwords, BOW, 10K features, negation	0.85	0.89	0.76	0.83
Logistic Regression, stopwords, TF-IDF, 5K features, class_weight = balanced	0.84	0.88	0.77	0.83
Logistic Regression, stopwords, BOW, 5K features, class_weight = balanced	0.84	0.88	0.76	0.82
Logistic Regression, Word2Vec (Avg Pooling), Vector Size=100	0.81	0.85	0.73	0.79
Decision Tree, stopwords, TF-IDF, 5K features, class_weight = balanced, max_depth = 30	0.74	0.80	0.64	0.72
Decision Tree, stopwords, BOW, 5K features, class_weight = balanced, max_depth = 30	0.74	0.80	0.63	0.72
Random Forest, stopwords, TF-IDF, 5K features, class_weight = balanced, max_depth = 20, estimators = 100	0.83	0.87	0.73	0.80
Random Forest, stopwords, BOW, 5K features, class_weight = balanced, max_depth = 20, estimators = 100	0.82	0.87	0.71	0.79
SVM, stopwords, TF-IDF, 5K features, class_weight = balanced, max_iter = 2K	0.84	0.88	0.76	0.82
SVM, stopwords, TF-IDF, 15K features, class_weight = balanced, max_iter = 2K, negation	0.86	0.90	0.78	0.84
SVM, stopwords, BOW, 5K features, class_weight = balanced, max_iter = 2K	0.83	0.87	0.74	0.81
SVM, stopwords, Word2Vec (Avg Pooling), 5K features, class_weight = balanced, max_iter = 2K	0.81	0.85	0.73	0.79
DistilBERT (fine-tuning), AutoTokenizer, learning_rate=2e-5, train_batch_size=16, train_epochs=3	0.92	0.95	0.87	0.91
GPT-4o (Zero-shot Generic)	0.90	0.89	0.91	0.90
GPT-4o (Zero-shot Domain)	0.92	0.91	0.93	0.92
GPT-4o (Few-shot)	0.90	0.89	0.91	0.90

5.1 Analysis and Discussion

One of the aspects that seemed clearest to us, before carrying out any analysis, was the need to use a metric other than accuracy. Since our dataset has many more positive reviews than negative ones, accuracy hides the models' poor performance in classifying the minority class (in this case, the negative reviews). Due to limited space, and the already large size of our table (Table 1), we do not present the recall and precision for each class, but we can use the TextBlob results as an example: An

accuracy of 77% as a baseline looks excellent at first glance and the model achieved very good results in predicting positive reviews. If we look at the negative class, it has a precision of 0.74, very close to the 0.71 for the positive class, but it has a recall of 0.39 (0.94 for the positive class). This means that the model fails to detect the vast majority of negatives. Moreover, if instead of using a model we predicted that all reviews in the test set were positive, we would end up with an accuracy of 70% (the rounded percentage of positive reviews in the set). While not perfect, we considered the Macro F1-Score to be a more honest metric, giving equal importance to the results of the minority class and the majority class.

For the lexicon- and rule-based models, VADER, which was created mainly to perform well on social media documents, ended up exceeding our expectations within its category, while STANZA, by contrast, was surprising for its poor results, given that it is a neural network-based model. One of the hypotheses we put forward for this poor performance is that STANZA may have assigned polarity to each review much more conservatively, assigning 0 polarities much more often (which, in our solution, are labeled as negative) than VADER. One piece of evidence supporting this argument is found in the negative-class-specific results of both: For VADER, 0.78 precision and 0.44 recall (most negatives are not “detected”), while for STANZA we have 0.44 precision and 0.82 recall (STANZA considers almost everything negative). These models clearly have difficulty predicting the negative class.

The polarity-sum model with the NCR EmoLex lexicon performed even worse, which was to be expected, as it is a considerably simpler solution. Negation handling had a clear effect on its ability to classify negative reviews, raising the F1-Score from 0.38 to 0.49. In addition, the use of Valence shifting inspired by the study by Hogenboom et al had no effect at all on the model's results. We attribute this to the fact that EmoLex is a lexicon that assigns a binary polarity to its classes, with no words being “more positive” than others, unlike the lexicon used in the work referred to. Thus, in a review, the difference in total polarity, the additional weight given to positive terms under the effect of negation is inconsequential (-0.27 additional polarity points per negated positive term compared with the model without this treatment).

In the classical trained-model approaches, it is worth highlighting the leap in performance quality compared with the previous models. The large size of the training set compared with the test set certainly helped to create models with high predictive ability, as did the use of class_weight = “balanced”, since this additional weight given to the minority class was reflected in the increase in the F1-Score of the negative class. Negation handling slightly improved the results in all models, but what deserves most attention is the already high performance of models with simple data treatment, such as Naive Bayes with BOW. The major shortcoming of Naive Bayes with TF-IDF stems from the very way the model works (TF-IDF normalizes term frequencies, whereas the model is built with raw discrete term frequencies in mind, which explains its performance with BOW), which, in hindsight, seems to make this experiment unnecessary. As for the SVM experiments, TF-IDF vectorization works better than BOW (in this case, TF-IDF normalization is a positive aspect). The highest Macro F1-Score in this section belongs to SVM with TF-IDF and negation handling; however, it is difficult to attribute this success solely to the data treatment,

since we tripled the number of features in the model (for future work, the success of negation should be put to the test with 10K features, thus making it comparable to the Naive Bayes experiment). We also highlight Logistic Regression with TF-IDF.

The DistilBERT SST2 model achieved results competitive with the classical models trained on the specific domain of the data, but fine-tuning the original DistilBERT model on the training set showed what we consider an impressive predictive ability, being beaten only by the use of LLMs (in this case, GPT 4-o) with Zero-shot Domain, the latter having been tested on a tiny number of reviews compared with the other models.

Finally, we believe that the drop in performance of the GPT 4-o few-shot experiment was caused by a lack of quality in its Prompt Engineering.

6 Conclusions

Although the great capability of (commercial) LLMs across all kinds of tasks is clear, we consider that the barrier to using them with large volumes of data without a huge budget (and even with such a budget, time limits imposed by the provider remain) makes this method unfeasible for the overwhelming majority of large-scale data classification cases. We therefore conclude that, among the models evaluated, fine-tuned DistilBert delivers the best performance, taking computational cost into account, in sentiment classification of Amazon food product reviews. We further conclude that trained classical Machine Learning models, for this same purpose, are able to remain competitive, notably the SVM model with TF-IDF and negation handling (15K features) and Logistic Regression with TF-IDF (5K features).

For future work, in addition to what has already been mentioned in section 5, we recommend testing negation handling for Logistic Regression with TF-IDF, since we believe it may outperform the similar experiment with the SVM model.

We recommend developing new fine-tuning experiments with the DistilBERT model, which may further improve its performance.

It would be important, given the necessary computational capacity, to test the performance of local offline versions of LLMs, since, if they performed comparably to the full versions, their use would become feasible.

References

1. Gope, J.C., Tabassum, T., Mabrur, M.M., Yu, K., Arifuzzaman, M.: Sentiment analysis of Amazon product reviews using machine learning and deep learning models. In: 2022 International Conference on Advancement in Electrical and Electronic Engineering (ICAEEE)
2. Hawlader, M., Ghosh, A., Raad, Z.K., Chowdhury, W.A., Shehan, M.S.H., Ashraf, F.B.: Amazon product reviews: Sentiment analysis using supervised learning algorithms. In: 2021 International Conference on Electronics, Communications and Information Technology (ICECIT), pp. 1–6. IEEE (2021)
3. Lu, J.: Text vectorization in sentiment analysis: A comparative study of TF-IDF and Word2Vec from Amazon Fine Food Reviews. ITM Web of Conferences, vol. 70,

03001. EDP Sciences (2025). DOI: 10.1051/itmconf/20257003001
4. Tabany, M., Gueffal, M.: Sentiment Analysis and Fake Amazon Reviews Classification Using SVM Supervised Machine Learning Model. Journal of Advances in Information Technology, vol. 15, no. 1, pp. 49–58 (2024). DOI: 10.12720/jait.15.1.49-58
5. Ali, H., Hashmi, E., Yildirim, S.Y., Shaikh, S.: Analyzing Amazon Products Sentiment: A Comparative Study of Machine Learning, Deep Learning, and Transformer-Based Techniques. Electronics, vol. 13, no. 7, 1305 (2024)
6. Maurya, R., Pratap, A.: Sentiment Analysis on Amazon Product Reviews. International Journal of Computer Science and Engineering, vol. 6, no. 5, pp. 998–1003 (2018)
7. Loria, S. (2018). TextBlob: Simplified Text Processing. [Online]. Available at: https://textblob.readthedocs.io/
8. Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. In Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM-14)
9. Qi, P., Zhang, Y., Zhang, Y., Bolton, J., & Manning, C. D. (2020). Stanza: A Python Natural Language Processing Toolkit for Many Human Languages
10. Hugging Face: distilbert-base-uncased-finetuned-sst-2-english. Hugging Face Hub (2024). DOI: 10.57967/hf/0181. Available at: Hugging Face model page
11. Mohammad, S. M., & Turney, P. D. (2013). Crowdsourcing a Word-Emotion Association Lexicon. Computational Intelligence, 29(3), 436-465
12. Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media, Inc.
13. Hogenboom, A.; van Iterson, P.; Heerschop, B.; Frasincar, F.; Kaymak, U. Determining negation scope and strength in sentiment analysis. 2011 IEEE International Conference on Systems, Man and Cybernetics - SMC. LOCATION OF CONFERENCE, United StatesDATE OF CONFERENCE; pp. 2589–2594.
14. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830
15. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space
16. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter